
Trolley Retrieval: A Complete, Machine-Checked Solution

Akaash Dash
akaash.dash@gmail.com

Abstract

We give a complete, machine-checked solution to the *Trolley Retrieval* problem (G-Research GRiddles Series A, Puzzle 4). Writing c for the permutation of the positive integers that labels the tape, the answer is

Bob has a winning strategy $\iff c$ is *not* eventually the identity

that is, $c(n) \neq n$ for infinitely many n (equivalently, c is not finitary). The losing direction is a parity lock that makes consecutive cells permanently indistinguishable once c has an identity tail. The winning direction builds an explicit, signal-independent exploration strategy, splitting on whether the displacement $c(n) - n$ is bounded above: a single fixed “lag” recurs when it is, whereas in the unbounded-above regime the natural single-lag recurrence is *provably false*, so a growing-lag staircase tailored to c is used instead. Every step is formalised in **Lean 4**: the main theorem is fully sorry-free and axiom-clean, its `#print axioms` audit listing only the three standard foundational axioms (`propext`, `Classical.choice`, `Quot.sound`).

1 The problem

The exact statement is as follows. Let $c(1), c(2), \dots$ be a sequence of positive integers in which every positive integer appears exactly once; equivalently c is a permutation (bijection) of $\mathbb{N}^+$. A one-way infinite sequence of cells is given: cell n (the n -th from the left, $n \geq 1$) is labelled $c(n)$.

At time $t = 0$ an invisible trolley is placed on an unknown cell, the k_0 -th from the left. For each integer $t \geq 1$, in order:

1. Bob chooses a direction, left or right; the trolley moves one cell that way, to cell k_t .
2. The trolley sends **Yes** if $c(k_t) < t$, and **No** otherwise.

If ever $k_t = 0$ (the trolley falls off the left end), Bob loses immediately. At any time Bob may instead *guess* the trolley’s position; a correct guess retrieves it, an incorrect guess loses.

Determine all permutations c for which Bob has a strategy guaranteeing safe retrieval.

Bob knows c in full; the only hidden datum is the start cell k_0 . The trolley is invisible—Bob never sees k_t directly. His sole feedback is the binary signal, and he chooses each move adaptively from the signal history.

1.1 The answer

Theorem 1.1 (Main). *Bob has a strategy ensuring safe retrieval if and only if c is not eventually the identity; that is, if and only if $c(n) \neq n$ for infinitely many n .*

Bob wins from every start cell $\iff c(n) \neq n$ for infinitely many n .

We say c is *eventually identity* (EI) if there is a threshold N with $c(n) = n$ for all $n > N$. Since c is a bijection, “not EI” has several equivalent forms, all used below:

- $\{n : c(n) \neq n\}$ is infinite (c is not finitary);
- c^{-1} is not eventually identity (the condition is symmetric under $c \leftrightarrow c^{-1}$);
- c has infinitely many *upward gaps*: positions p with $c(p+1) - c(p) \geq 2$.

The last equivalence is worth recording, as it is exactly what fails in the EI case.

Proposition 1.2. *c has infinitely many upward gaps if and only if c is not eventually identity.*

Proof. If $c(n+1) \leq c(n) + 1$ for all $n > N$ (no gaps past N), then on (N, ∞) the bijection c is non-decreasing by steps of at most 1; a value-preserving bijection of a tail of $\mathbb{N}^+$ that never jumps up by 2 cannot skip a value and cannot repeat one, forcing $c(n) = n$ for all $n > N$. The contrapositive is the claim. $\square$

1.2 The game model

We record the model precisely, since the formalisation depends on it. Positions are integers; the start cell $k_0 \in \mathbb{N}^+$ is a positive integer but a trajectory may, in principle, drive a position to ≤ 0 , which loses. A *strategy* σ maps a signal history (most-recent first) to an *action*: either a move (left/right) or a guess of a cell. From a start k_0 , the strategy determines a sequence of states (position _{t} , history _{t}); the signal appended at time t is **Yes** iff the (valid) new position p satisfies $c(p) < t$. *Bob wins from k_0* if there is a finite stop time T such that (i) every visited position through T is a valid cell (≥ 1), (ii) every action before T is a move, and (iii) at time T the strategy guesses exactly the current position $k_0 + D_T$, where D_T is Bob’s net displacement. *Bob wins* (BobWins c) if a single strategy wins from every start cell.

1.3 The alarm-clock view and the parity lock

It is convenient to read the labels as wake-up times. Think of cell n as carrying an alarm clock set for time $c(n)$: at every time $t > c(n)$ the cell is “awake”, and a trolley standing on it reports **Yes**; while $t \leq c(n)$ the cell is “asleep” and reports **No**. Because c is a bijection, exactly one cell wakes at each integer time, and every cell wakes eventually. This is just a restatement of signal = **Yes** $\iff c(k_t) < t$.

Let $D_t := k_t - k_0$ be Bob’s net displacement at time t .

Lemma 1.3 (Parity lock). *As long as Bob has not yet guessed, $D_t \equiv t \pmod{2}$. Equivalently, $t - D_t$ is always even.*

Proof. $D_0 = 0$, and each move changes D by ± 1 and t by $+1$, so the parity of $t - D_t$ is invariant; it starts even. $\square$

The parity lock is the single structural fact behind the losing direction. A second, equally elementary identity exposes *why*. Let L_t be the number of *left* moves Bob has made by time t and R_t the number of right moves, so $t = L_t + R_t$ and $D_t = R_t - L_t$. Then

$$t - D_t = (L_t + R_t) - (R_t - L_t) = 2L_t. \quad (1)$$

Write $g(m) := c(m) - m$ for the displacement of the label from the diagonal. The signal at time t is

$$\begin{aligned} \text{Yes} &\iff c(k_0 + D_t) < t \iff g(k_0 + D_t) < t - D_t - k_0 \\ &\iff g(k_0 + D_t) < 2L_t - k_0. \end{aligned} \quad (2)$$

In other words, the walk reads the sequence g against a threshold $2L_t - k_0$ that Bob controls only through his *left-move count*.

Remark 1.4 (The whole obstruction, in one line). *For the identity, $g \equiv 0$, so (2) collapses to $\text{Yes} \iff k_0 < 2L_t$. Bob can only ever compare k_0 against even thresholds, and $2j < 2L \iff 2j+1 < 2L$ for every L . Hence the start cells $2j$ and $2j+1$ produce identical signals under every strategy: the parity bit of k_0 is unrecoverable. This is the seed of the losing direction; the work below is to extend it to every eventually-identity c despite the perturbed initial segment.*

2 Losing direction: eventually identity $\Rightarrow$ Bob loses

Theorem 2.1 (EventuallyIdentity_loses). *If c is eventually identity, Bob has no strategy that wins from every start cell.*

The proof isolates a structural “parallel-evolution” lemma and then verifies its hypothesis for two carefully chosen start cells.

2.1 Parallel evolution

Fix a strategy σ . For a start cell k let $\text{pos}(k, t)$ be the position and $\text{hist}(k, t)$ the signal history (most-recent first) at time t . Bob’s action at any decision point is a function of hist alone.

Lemma 2.2 (Parallel evolution; `parallel_evolution`). *Let k_1, k_2 be two start cells. Suppose that, started from k_1 , Bob makes no guess before time t , and that at every time $s \leq t$ the signal observed from k_1 ’s trajectory equals the signal observed from k_2 ’s trajectory. Then the two histories agree at time t , and the positions stay rigidly offset:*

$$\text{hist}(k_1, t) = \text{hist}(k_2, t), \quad \text{pos}(k_2, t) = \text{pos}(k_1, t) + (k_2 - k_1).$$

Proof. Induction on t . At $t = 0$ the histories are empty and the offset is $k_2 - k_1$ by definition. For the step, equal histories make σ choose the *same* action in both worlds; since Bob has not guessed, that action is a move, applied identically to both, preserving the offset. The new signal entries are equal by the time- $(t+1)$ hypothesis, so the histories remain equal. $\square$

Corollary 2.3. *If $k_1 \neq k_2$ and the two signal traces agree at every time under σ , then σ cannot win from both k_1 and k_2 .*

Proof. Equal traces force equal histories at all times (Lemma 2.2 applied up to either guess time), hence equal first-guess times $T_1 = T_2 =: T$ and equal guesses $g_1 = g_2$. But correctness would require $g_1 = \text{pos}(k_1, T)$ and $g_2 = \text{pos}(k_2, T)$, while $\text{pos}(k_2, T) = \text{pos}(k_1, T) + (k_2 - k_1)$. These contradict $k_1 \neq k_2$. $\square$

2.2 The indistinguishable pair for eventually-identity c

Suppose $c(n) = n$ for all $n > N$. Let M be an upper bound for the finitely many perturbed labels: $M \geq \max\{c(m) : 1 \leq m \leq N\}$ and $M \geq N + 1$. Choose j with $2j > M$ and set

$$k_1 = 2j, \quad k_2 = 2j + 1.$$

Lemma 2.4 (Signal match; `signal_match`). *Fix a no-guess strategy run from $k_1 = 2j$. At every time t , every position $p \geq 1$ reachable from $2j$ in t moves (so $2j - t \leq p$) that respects the parity lock ($t + p$ even) satisfies*

$$\text{signal at } p \text{ at time } t = \text{signal at } p+1 \text{ at time } t.$$

Proof. Two cases on the location p .

(a) $p > N$ (identity region). Both p and $p+1$ exceed N , so $c(p) = p$ and $c(p+1) = p+1$. The signal at p is **Yes** $\iff p < t$, and at $p+1$ is **Yes** $\iff p+1 < t$. These differ only if $t = p + 1$. But then $t + p = 2p + 1$ is odd, violating the parity hypothesis $t + p$ even. So the signals agree.

(b) $p \leq N$ (perturbed region). Here the trolley has travelled from $2j$ to $p \leq N$, so $t \geq 2j - p \geq 2j - N$; concretely $2j > M \geq N + 1$ forces $t > M$. Both $c(p)$ and $c(p+1)$ are at most M (using $c(N+1) = N+1 \leq M$ for the boundary), hence $c(p), c(p+1) < t$: both signals are **Yes**. They agree. $\square$

The two cases capture the dichotomy of Remark 1.4: in the identity tail, the parity lock makes $2j$ and $2j+1$ collide; in the perturbed segment, the time cost of reaching it ($t \geq 2j - N$) is so large that every cell there is already awake, so both worlds report **Yes** and again collide.

Proof of Theorem 2.1. Suppose σ wins from every start cell, in particular from $k_1 = 2j$ and $k_2 = 2j+1$ above. We claim the two signal traces agree at every time, which contradicts Corollary 2.3.

By induction on t , the trajectories from $2j$ and $2j+1$ evolve in lockstep with positions offset by exactly 1 and equal histories, as long as Bob has not guessed: the inductive step applies the signal match (Lemma 2.4) at $\text{pos}(2j, t+1)$, whose hypotheses—validity $p \geq 1$ (Bob wins, so the trajectory is safe), reachability $2j - t \leq p$ (a ± 1 walk), and parity (Lemma 1.3)—all hold. Hence the signal entering both histories at each step is identical, so $\text{hist}(2j, t) = \text{hist}(2j+1, t)$ and $\text{pos}(2j+1, t) = \text{pos}(2j, t) + 1$ for all t . The traces agree at every time, and Corollary 2.3 gives the contradiction. $\square$

Remark 2.5 (The half-infinite tape is irrelevant here). *For the chosen large $k_0 \approx 2j$, no finite-time strategy can reach cell 0, so the “falling off” rule never binds; the indistinguishability argument is unaffected by the absence of a left end.*

3 Winning direction: not eventually identity $\Rightarrow$ Bob wins

The winning direction has three layers, treated in Sections 3.1–3.4. First, a *distinguishability theorem*: when c is not eventually identity, no pair of start cells can stay indistinguishable. Second, a generic *reduction*: any single signal-independent trajectory that is safe, informative, and pairwise-separating is already a winning strategy. Third, an explicit construction of such a trajectory, which splits into two regimes according to whether c ’s displacement is bounded above.

3.1 The distinguishability theorem

Reaching displacement D from the start costs time at least $|D|$, and the parity lock forces $t \equiv D \pmod{2}$. Within those constraints Bob can pad with oscillation to reach any admissible time, and (because a cell once awake stays awake) the only information at displacement D is the *set of times $t \geq D$, $t \equiv D \pmod{2}$, at which the cell is awake*—equivalently, the single comparison $c(a + D) < t$. This motivates:

Definition 3.1 (Indistinguishable pair; `Indistinguishable`). *Two start cells $a < b$ are indistinguishable (for c) if for every displacement $D \geq 0$ and every time $t \geq D$ with $t \equiv D \pmod{2}$,*

$$c(a + D) < t \iff c(b + D) < t.$$

Quantifying over *all* reachable (D, t) —including small times that catch early wake-ups—is essential; it is exactly the freedom that lets Bob’s oscillation observe a cell’s wake-up time at full resolution.

Theorem 3.2 (`Distinguishability`; `Indistinguishable_implies_eventually_identity`, `not_eventually_identity_implies_distinguishable`). *If $a < b$ are indistinguishable for c , then $b = a + 1$ and $c(n) = n$ for all $n \geq a$. Consequently, if c is not eventually identity, then no pair of start cells is indistinguishable.*

We prove the first sentence; the second is its contrapositive (an indistinguishable pair would exhibit an identity tail, making c EI). Throughout write $\Delta := b - a \geq 1$.

The proof has two regimes by the size of Δ . The case $\Delta = 1$ is a clean shift argument; the case $\Delta \geq 2$ is the crux. We first extract the local structure shared by both.

Lemma 3.3 (`Consecutiveness`; `indist_consec`). *Call a displacement D active (for the cell a) if $c(a + D) \geq D$, and similarly for b . If $a < b$ are indistinguishable and D is active for both a and b , then $c(a + D)$ and $c(b + D)$ are consecutive integers: $|c(a + D) - c(b + D)| = 1$.*

Proof. Let $X = c(a + D)$, $Y = c(b + D)$. Injectivity gives $X \neq Y$; say $X < Y$. If $Y \geq X + 2$, pick t with $X < t \leq Y$ of the parity $t \equiv D \pmod{2}$ (possible because the interval $(X, Y]$ has length ≥ 2 and so contains both parities). Then $c(a + D) < t$ but $c(b + D) \not< t$, with $t \geq D$ (as D is active and $t > X \geq D$), contradicting indistinguishability at (D, t) . Hence $Y = X + 1$. The case $Y < X$ is symmetric. $\square$

Lemma 3.4 (Bad displacements are forward-closed; `bad_D_propagates`). *Call D bad (for a) if $c(a+D) < D$. If $a < b$ are indistinguishable and D is bad, then $D+\Delta$ is bad: $c(a+D+\Delta) < D+\Delta$.*

Proof. D bad means the cell $a + D$ is awake at the least admissible time $t \leq D + 1$ with $t \equiv D \pmod{2}$, so its signal is **Yes**. Indistinguishability transfers **Yes** to $b + D = a + (D + \Delta)$: $c(a + D + \Delta) < t \leq D + 1$, and since the value is an integer $c(a + D + \Delta) \leq D < D + \Delta$. (The point is purely that an already-saturated **Yes** propagates by $+\Delta$; this is where the constraint $b = a + \Delta$ enters.) $\square$

Lemma 3.4 is decisive: on each residue class modulo Δ , the bad displacements form an *up-set*, so the *active* (non-bad) displacements form a finite or cofinite *prefix*—never an interleaving of good and bad. With it, the crux is short.

Lemma 3.5 (Chain bound; `chain_bound`). *If $a < b$ are indistinguishable, fix a residue r with $0 \leq r < \Delta$. For any k such that $r, r + \Delta, \dots, r + k\Delta$ are all active, $c(a + r + k\Delta) \leq c(a + r) + k$.*

Proof. By Lemma 3.3, consecutive active displacements on the residue satisfy $|c(a + r + (i+1)\Delta) - c(a + r + i\Delta)| = 1$ (apply consecutiveness at the displacement $r + i\Delta$, whose $+\Delta$ shift lands at $b + r + i\Delta$). Summing k unit steps and the triangle inequality give $c(a + r + k\Delta) \leq c(a + r) + k$. $\square$

Lemma 3.6 ($\Delta \geq 2$ is impossible; `indist_Delta_ge_2_impossible`). *If $a < b$ are indistinguishable with $\Delta = b - a \geq 2$, contradiction.*

Proof. Let $M := \max_{0 \leq r < \Delta} c(a + r)$. Suppose a displacement D is active, and write $D = r + k\Delta$ with $0 \leq r < \Delta$. By forward-closure (Lemma 3.4), all of $r, r + \Delta, \dots, r + k\Delta$ are active (active is a prefix on each residue). Activity gives $c(a + D) \geq D = r + k\Delta$, while the chain bound gives $c(a + D) \leq c(a + r) + k \leq M + k$. Hence

$$r + k\Delta \leq M + k \implies k(\Delta - 1) \leq M - r \leq M.$$

With $\Delta \geq 2$ this bounds $k \leq M$, so $D = r + k\Delta < (M+1)\Delta$. Thus *every* displacement $D \geq (M+1)\Delta$ is bad: $c(a + D) < D$ for all such D .

This forces a pigeonhole collapse. For $D \geq (M+1)\Delta$, the cell at position $a + D$ has label $c(a + D) < D = (a + D) - a$, i.e. $c(p) < p - a$ for all large p . The finitely many small positions have labels bounded by some constant. Choosing N large enough, c maps the segment $[1, N]$ into $[1, N - a - 1]$: large positions p land below $p - a \leq N - a - 1$, and the bounded small positions also fit once N is large. But $[1, N]$ has N elements and $[1, N - a - 1]$ has $N - a - 1 < N$, so c cannot be injective on $[1, N]$. Contradiction. $\square$

Remark 3.7 (Why this is the right proof). *The contradiction in Lemma 3.6 uses no parity argument and no even/odd split on Δ . The single lever is forward-closure of badness (Lemma 3.4), which turns the “active” set on each residue into a prefix; the chain bound then makes that prefix finite for $\Delta \geq 2$, and a one-line pigeonhole finishes. This is markedly simpler than the even/odd case analysis we initially attempted, and it was the breakthrough that turned the crux from “research-level” into elementary.*

It remains to handle $\Delta = 1$.

Lemma 3.8 ($\Delta = 1$ forces an identity tail; `shift_one_implies_eventually_identity`). *If $a < b = a + 1$ are indistinguishable, then $c(n) = n$ for all $n \geq a$.*

Proof. For $\Delta = 1$, $b + D = a + (D + 1)$, so consecutiveness (Lemma 3.3) on active displacements reads $|c(a + D + 1) - c(a + D)| = 1$: the labels along $a, a+1, a+2, \dots$ form a self-avoiding ± 1 walk. Let $\sigma(0) = \text{sgn}(c(a+1) - c(a))$ be the sign of the first step. If $\sigma(0) = -1$ (descending start), the walk $c(a), c(a)-1, c(a)-2, \dots$ would eventually drop below 1, impossible for labels in $\mathbb{N}^+$; so $\sigma(0) = +1$. A self-avoiding ± 1 walk that starts ascending must ascend forever (any later descent would revisit a value, breaking injectivity), giving $c(n+1) = c(n) + 1$ for all $n \geq a$, i.e. $c(n) = n + C$ on $[a, \infty)$ with $C = c(a) - a$. If $C > 0$, the values $\{1, \dots, C\}$ are absent from $c([a, \infty))$ and must be supplied by the finite set $c([1, a-1])$, which has only $a-1$ elements; bijectivity of c on $\mathbb{N}^+$ then forces $C = 0$ (and $C < 0$ was already excluded). Hence $c(n) = n$ for $n \geq a$. $\square$

Proof of Theorem 3.2. Let $a < b$ be indistinguishable, $\Delta = b - a$. By Lemma 3.6, $\Delta \geq 2$ is impossible, so $\Delta = 1$, i.e. $b = a + 1$; then Lemma 3.8 gives $c(n) = n$ for $n \geq a$, so c is eventually identity. Contrapositively, if c is not eventually identity then no pair is indistinguishable. $\square$

3.2 The reduction: separation + safety + Yes $\Rightarrow$ Bob wins

Theorem 3.2 says only that every pair of cells admits *some* separating observation. Turning this into a single winning strategy is the substantial part of the problem. The key conceptual move is that Bob does not need an *adaptive* (signal-dependent) move schedule at all: a fixed, signal-independent exploration trajectory suffices, provided it is tailored to c . He plays the trajectory blind, using the signals only to decide *when to stop and guess*.

Fix a deterministic move sequence $m: \mathbb{N} \rightarrow \{L, R\}$ and write $\text{pos}_m(k_0, t)$ for its position from start k_0 . The strategy `ofMoves` m replays m 's moves blindly. Call the trajectory:

- *safe* if $\text{pos}_m(k_0, t) \geq 1$ for all k_0, t (never falls off the left);
- *Yes-emitting* if from every k_0 some time emits **Yes**;
- *separating* if for every $a \neq b$ the signal histories from a and from b eventually differ.

Theorem 3.9 (Reduction; `localizes_BobWins`, `ofMoves_localizes`). *If a fixed trajectory m is safe, Yes-emitting, and separating, then Bob wins from every start cell.*

The nontrivial point is that separation is an *infinite* family of conditions (one per wrong candidate), whereas a win requires a *single finite* stop time T that has ruled out *all* wrong candidates at once. The bridge is first-Yes finiteness.

Lemma 3.10 (First-Yes finiteness; `yesSet_finite`). *For any time s and displacement-offset δ , the set of start cells k whose signal at time s (after the shift δ) is **Yes** is finite.*

Proof. A **Yes** at (s, δ) from start k means $c(k + \delta) < s$. As c is a bijection, only the finitely many values $\{1, \dots, s - 1\}$ lie below the threshold, so only finitely many cells $k + \delta$ (hence finitely many k) can emit **Yes** at that moment. $\square$

Proof of Theorem 3.9. Fix the true start k_0 . By Yes-emission there is a first time s_0 at which k_0 's trajectory reads **Yes**. Any candidate k *not yet distinguished* from k_0 by time s_0 must read the *same* signal, hence also **Yes**, at s_0 ; by Lemma 3.10 there are only finitely many such k . Each of these finitely many candidates is separated from k_0 at *some* finite time (separation), so the maximum T of those finitely many separation times is finite, and by time T *every* wrong candidate has produced a differing signal. Bob plays m until T ; the histories now pin k_0 uniquely. Knowing k_0 and his own (signal-independent) displacement D_T , Bob computes the current cell $k_0 + D_T$ and guesses it. Safety guarantees every visited position was a valid cell, so the guess is timely and correct. $\square$

In Lean, the “stop and guess” wrapper is the belief-state strategy `beliefStrat`: it replays m 's moves until the set of signal-consistent candidates is a singleton, then guesses. The match between the wrapped and bare trajectories up to the first resolved time, and the correctness of the guess, are discharged in `localizes_BobWins`. *All trajectories we construct keep $D \geq 0$ at all times*, so safety is automatic: $\text{pos} = k_0 + D \geq k_0 \geq 1$. What remains is to exhibit one fixed trajectory that is separating and Yes-emitting.

3.3 Lag monotonicity and why a single fixed lag fails

The trajectory's structure is governed by one invariant. Define the *lag* $\ell_t := t - D_t = 2L_t$ (twice Bob's left-move count, by (1)). Since each move changes D by ± 1 and t by $+1$,

$$\ell_{t+1} - \ell_t \in \{0, 2\} :$$

the lag is **monotone nondecreasing and never returns to a smaller value**. By the alarm-clock identity (2), a separating observation for a pair (a, b) at displacement D lives at a definite even *slack* $s = t - D = \ell_t$, namely the event

$$[c(a + D) < D + s] \neq [c(b + D) < D + s].$$

Because the lag only increases, a separator at slack s is *use-it-or-lose-it*: once Bob's lag has passed s , he can never again present that pair with that slack. He must therefore visit separators in nondecreasing slack order. This single fact dictates the whole construction.

The natural hope is that, for each pair (a, b) , separators *recur* at one *fixed* even slack $s(a, b)$: there are infinitely many displacements D at which (a, b) is separated at the *same* slack s . If that held universally, Bob could fix a slack per pair and ride right to catch a recurrence. We call this the *band recurrence*. It is tempting but **false**.

Remark 3.11 (The band recurrence is false: the AP-split counterexample). *Consider the permutation that splits $\mathbb{N}^+$ into an arithmetic-progression shape: on the even cells, $c(2k) = 2k + 2\lfloor\sqrt{k}\rfloor$ (the even labels ascend slowly, with displacement $g \rightarrow \infty$), and the odd cells fill the remaining values in increasing order (their displacement drifts to $-\infty$). This is a genuine non-eventually-identity bijection. For the pair $(2, 4)$ the separating slack climbs like $\sqrt{D}$: at even displacement D the separating window is roughly $(2 + 2\sqrt{D}, 4 + 2\sqrt{D})$, of width ≈ 2 , escaping upward. Any fixed even slack s sits inside that window only for D in a bounded range and is then abandoned forever as the window climbs. Hence no fixed slack recurs at unbounded displacement, and the band recurrence fails. Bob still wins—the separators exist, they merely sit at a growing slack—but only a trajectory that raises its lag without bound can catch them. This is the concrete realisation of the principle that no single fixed move schedule is universal, and it motivates the dichotomy of the next section.*

3.4 The dichotomy: a c -tailored trajectory in each regime

Whether the band recurrence holds turns out to be governed by a single structural property of c . Say c is *bounded above* if there is a constant B with $c(n) \leq n + B$ for all n (equivalently, $g(n) = c(n) - n$ is bounded above). The main theorem's winning direction dispatches on this property (main's `by_cases` in `Answer.lean`):

- **Bounded above.** The band recurrence *holds*: each pair has a fixed recurring even slack, and a lag-monotone *fixed-lag staircase* catches it (Section 3.4.1).
- **Unbounded above.** The band recurrence may fail (Remark 3.11), but separators exist at *arbitrarily large* even slack, and a c -tailored *growing-lag staircase* visits one per pair, raising its lag monotonically (Section 3.4.2).

In both regimes the trajectory is a single, predetermined, signal-independent move sequence with $D \geq 0$ throughout, fed to the reduction (Theorem 3.9). No adaptive belief-state strategy is needed.

3.4.1 Bounded above: the fixed-lag staircase

The combinatorial input here is the band recurrence at a max-bounded slack.

Theorem 3.12 (Band recurrence under bounded displacement; `band_of_boundedAbove`). *If c is not eventually identity and is bounded above, then for any distinct a, b there is an even slack $s \geq \max(a, b)$ whose slack- s separators recur at arbitrarily large displacement: $\forall D_0 \exists D \geq D_0$ with $[c(a+D) < D+s] \neq [c(b+D) < D+s]$.*

Proof idea. Work with the integer displacement $f(n) = c(n) - n$. If the band failed at every even slack $s \geq \max(a, b)$, then past some finite floor the two displaced signals would *agree* for all D (`signals_agree_of_no_separator`, via the window description $\min < D+s \leq \max$). Boundedness above, combined with infinitely many weak ascents (`weak_ascents_infinite`), pins a *top recurring value* $v^* \geq 0$ for f . If $v^* = 0$ then $f(n) \leq 0$ cofinitely, i.e. $c(n) \leq n$ for all large n , and the cut lemma below forces c eventually identity—contradiction. If $v^* \geq 1$, the agreement relation, propagated along the residue ray modulo $\Delta = b - a$, contradicts the recurrence of v^* . Either way the band holds. $\square$

The cut lemma used above is the elegant heart of this regime and is worth isolating.

Lemma 3.13 (Cut lemma; `descent_cofinite_imp_EI`). *If $c(n) \leq n$ for all $n \geq N$, then c is eventually identity.*

Proof. Let B bound c on the finite segment $\{1, \dots, N\}$. For any $M \geq \max(N, B)$, the descent hypothesis past N and the bound below N together show c maps $\{1, \dots, M\}$ into itself; being an injection of a finite set into itself, it is *onto* $\{1, \dots, M\}$. Hence $c(M+1)$ cannot land in $\{1, \dots, M\}$ (those values are already hit), so $c(M+1) > M$; with $c(M+1) \leq M+1$ from descent, c fixes $M+1$. Applying this for every $M = n-1$ past the threshold gives EI. (The ascent-cofinite mirror follows by applying this to c^{-1} .) $\square$

The staircase. Given the recurring slacks, the trajectory is assembled as a *band-top lag-monotone staircase* (`BandProvider`, `BobWins_of_bandProvider`). Enumerate the ordered distinct pairs (a, b) , $a < b$, by the key $(s(a, b), \max(a, b), \min(a, b))$ in increasing order. Because $s(a, b) \geq \max(a, b)$, only finitely many pairs have any given slack (at most $\binom{S}{2}$ pairs have $s \leq S$), so the keys form a well-order of type ω : every pair has finitely many predecessors. For each pair in turn, the staircase (i) *raises* its lag from the current value to $s(a, b)$ by oscillating $D \in \{0, 1\}$ via R,L pairs (each pair bumps ℓ by 2 while returning D); (ii) *rides* right at fixed lag $s(a, b)$ until the displacement lands on a recurring separator, which exists by Theorem 3.12; (iii) *tails* right so displacements grow without bound across phases. Each step keeps $D \geq 0$, so the trajectory is safe; it is separating by construction and Yes-emitting because the lag $\rightarrow \infty$ and every cell has infinitely many weak descents (`weak_descents_infinite`).

3.4.2 Unbounded above: the growing-lag staircase

When c is unbounded above, the band recurrence can fail (Remark 3.11), so a fixed-lag ride is impossible. The replacement is a recurrence not at a fixed slack but at *arbitrarily large* even slack.

Theorem 3.14 (Separators at arbitrarily large even slack; `q2even`). *If c is unbounded above, then for every distinct $a < b$ and every prescribed lag floor L there exists an even slack $s \geq L$ and a displacement D with $\lfloor c(a+D) \rfloor < D+s \rfloor \neq \lfloor c(b+D) \rfloor < D+s \rfloor$.*

Proof idea. Suppose not: for some floor L no even slack $s \geq L$ separates at any D (the “bad adversary”). Writing the separation window at displacement D as $(\min v_a, v_b, \max v_a, v_b]$ with $v_a = a + g(a+D)$, $v_b = b + g(b+D)$, the adversary says this window contains no even integer $\geq L$. Whenever the window’s top exceeds L , this forces it to have *width exactly 1* with an *odd* top (`width1_of_no_even_in_window`); width 1 at a link means $|c(a+D) - c(b+D)| = 1$, which (since $b+D = a+(D+\delta)$, $\delta = b-a$) shows g does not decrease going left by δ at any high link. Unboundedness above provides, for any M , a *spike* cell on the a -ray with $g \geq M$ (`phi_spike_on_a_ray`). Walking left in steps of δ from the spike (`leftward_walk`), g stays $\geq M$ all the way down to a fixed *chain-start* cell in $\{a, \dots, a+\delta-1\}$. But g on those finitely many fixed cells is bounded by a constant; choosing M above that constant yields a contradiction. Hence the bad adversary is impossible and the separators exist at every lag floor. (A residue pigeonhole over the δ chains makes the spike-to-chain-start step fully rigorous for all δ .) $\square$

The growing-lag staircase. The unbounded trajectory (`GrowStair.gmoves`) enumerates the pairs (`nthPair`) and processes them in turn. For the i -th phase it requests, via Theorem 3.14, a separator at an even slack *at least* the current lag plus a margin (the lag floor `Lfloor i`, which strictly exceeds all prior lags). It then walks/oscillates to raise the lag to that slack and rides right to the separator, always keeping $D \geq 0$. Because each phase’s requested slack exceeds the previous lag, the lags strictly increase across phases and the ω -enumeration of pairs is well-founded. Safety (`grow_safe`), separation (`grow_separates`), and Yes-emission (`grow_yes`) are proved exactly as before and fed to the reduction. This is the only place a *growing* lag is essential; it is what catches the climbing separators of the AP-split example (Remark 3.11).

Remark 3.15 (Why a fixed trajectory suffices after all). *Earlier we believed the winner had to be a genuinely adaptive belief-state machine, because no single fixed move schedule could reconcile catching low-slack discriminators (which want $D \approx t$) with bounding k_0 (which wants D small). The resolution is that the moves can be fixed (signal-independent) provided the trajectory is tailored to c : its slacks and ride lengths depend on c , but it is a single predetermined sequence, which is exactly what the reduction consumes. Lag monotonicity is then not an obstruction but an organising principle—the staircase simply visits separators in nondecreasing slack order.*

3.5 Two complete winning families (locally decodable)

Before the general construction, we record two natural families that win by a much simpler, fully self-contained strategy: a fixed-amplitude oscillation. These were the first parts of the winning direction we formalised, and they remain the cleanest worked instances.

Definition 3.16 (Locally K -decodable). c is locally K -decodable if the tuple of first-Yes times at displacements $0, 1, \dots, K-1$ —equivalently the bucketed tuple $(c(k_0), \dots, c(k_0 + K-1))$ —determines k_0 .

Theorem 3.17 (K -decodable winning theorems; `LocallyDecodable_BobWins` ($K=2$), `LocallyK3Decodable_BobWins` ($K=3$), `LocallyKDecodable_BobWins` (any K)). If c is locally K -decodable, Bob wins, via a period- $2K$ triangle-wave sweep of displacements $0, 1, \dots, K-1, \dots, 1, 0, \dots$ with K detectors (first Yes at each displacement). The displacement stays in $[0, K-1]$, so safety is automatic.

Example 3.18 (Adjacent transpositions; `adjPerm_BobWins`). Let c swap $2i-1 \leftrightarrow 2i$ for every i (so $c(n) = n+1$ if n is odd, $c(n) = n-1$ if n is even). Here $g(n) = c(n) - n \in \{+1, -1\}$ is bounded, yet c is not eventually identity; Bob wins by local decoding at $K=2$. This already shows the boundary is non-EI, not unbounded growth of g .

Example 3.19 (Forward block 3-cycle; `blkPerm_BobWins`). Let $c(m) = m+1$ unless $3 \mid m$, in which case $c(m) = m-2$ (each block $\{3b+1, 3b+2, 3b+3\}$ rotates forward). Then $c(m) \leq m+1$ always, so no rightward probe ever separates the candidates by navigation alone. But cell 3 wakes early, at time 2 (since $c(3) = 1$), which splits $k_0 = 1$ from $k_0 = 2$ at displacement 1: at $(D=1, t=3)$ the world $k_0 = 2$ stands on the already-awake cell 3 and reports Yes, while $k_0 = 1$ stands on cell 2, still asleep, and reports No. The $K=3$ sweep, monitoring small displacements from time 0, catches this early wake-up, so c is locally 3-decodable and Bob wins.

Remark 3.20 (The fixed- K architecture cannot cover all non-EI c). The locally- K -decodable classes (any fixed amplitude K , formalised in Lean for arbitrary K) are genuinely incomplete. The block-5 cycle—rotate each $\{5b+1, \dots, 5b+5\}$ forward—is non-EI, yet for every K some pair of start cells produces an identical K -tuple of first-Yes times: the colliding pair $(4, 5)$ has $c(4..9) = (5, 1, 7, 8, 9, 10)$ and $c(5..10) = (1, 7, 8, 9, 10, 6)$, which a coarse amplitude- K sweep cannot tell apart even though Theorem 3.2 guarantees them distinguishable (here at $D=0, t=2$: $c(4) = 5 \geq 2$ but $c(5) = 1 < 2$). A single fixed-amplitude trajectory cannot finely monitor all displacements at once. This is precisely why the general winning direction needs the c -tailored staircase of Section 3.4, whose lag varies with the pair.

3.6 Worked examples table

Permutation c	EI?	Bob	Reason
Identity $c(n) = n$	yes	loses	$(2j, 2j+1)$ collide: only even thresholds probeable (Rem. 1.4).
Finitary (any finite-support perm.)	yes	loses	EI with $N = \text{support bound}$; same collision for large k_0 .
Adjacent transpositions	no	wins	locally 2-decodable (Ex. 3.18); bounded g ; bounded-above staircase.
Forward block 3-cycle	no	wins	locally 3-decodable (Ex. 3.19); bounded above.
Parity-shift $c(2k)=2k+2, c(2k+1)=2k-1$	no	wins	bounded above ($g \in \{\pm 2\}$); fixed-lag staircase (§3.4.1).
AP-split (Rem. 3.11)	no	wins	unbounded above; growing-lag staircase (§3.4.2); no fixed lag works.
Any non-EI c	no	wins	dichotomy of §3.4 (Thm. 1.1).

Table 1: Outcomes by family. Eventually-identity $\iff$ Bob loses; the two non-EI regimes (bounded/unbounded above) are won by the corresponding staircase.

The two italicised non-EI rows are the instructive boundary cases. The *parity-shift* permutation has displacement $g(n) \in \{+2, -2\}$, constant on residues—a balanced, non-eventually-identity permutation that is bounded above; it is exactly the kind of example that defeats naive cut-counting arguments, yet the bounded-above staircase wins it with a single fixed lag. The *AP-split* permutation (Remark 3.11) is its unbounded-above counterpart: there *no* fixed lag recurs, and only the growing-lag staircase succeeds.

4 Machine-verified formalisation in Lean 4

To provide the highest degree of certainty, the entire argument is formalised and machine-checked in **Lean 4** (with Mathlib) in the development `griddles-p4-lean`. The headline is unconditional:

*The main theorem main, namely $\text{BobWins } c \leftrightarrow \neg \text{EventuallyIdentity } c$, is **fully sorry-free**, and its axiom audit*

```
#print axioms main = [propext, Classical.choice, Quot.sound]
```

lists only the three standard foundational axioms, with no sorryAx. The full library builds cleanly with `lake build` (3338 jobs).

“Axiom-clean” means precisely this: the `#print axioms audit` shows only `propext`, `Classical.choice`, and `Quot.sound`. Every component theorem below is individually axiom-clean, and so is their combination in `main`.

4.1 Module structure

The library is organised in dependency order:

1. **Defs** — Game semantics: `Perm`, `Strategy`, `Action`, the trajectory functions (state/position/history/displacement), and the predicates `WinsFrom`, `BobWins`, `EventuallyIdentity`.
2. **Parity** — The parity lock ($t - D_t$ even) and the position lower bound.
3. **Losing** — The entire losing direction: `parallel_evolution`, `signal_match`, and the headline `EventuallyIdentity_loses`.
4. **LemmaA** — The distinguishability theorem (Theorem 3.2), including the $\Delta \geq 2$ crux by forward-closure (`indist_Delta_ge_2_impossible`) and the $\Delta = 1$ shift (`shift_one_implies_eventually_identity`); the corollary `not_eventually_identity_implies_distinguishable`.
5. **WinningRestricted / WinningKProbe / WinningKProbeGen** — The locally K -decodable winning classes (Theorem 3.17) for $K=2$, $K=3$, and arbitrary K , with worked examples `adjPerm`, `cyc3Perm`, `blkPerm`.
6. **WinningAdaptive** — The reduction infrastructure: `separatorAtSlack` and `separatorAtSlack_iff_window`; the bijectivity engines `weak_descents_infinite` and `weak_ascents_infinite`; first-Yes finiteness (`yesSet_finite`); the reduction of `Moves` localizes and localizes `BobWins`; the `BandProvider` interface and the band-top staircase `BobWins_of_bandProvider`.
7. **BoundedAbove** — The cut lemma (Lemma 3.13): `descent_cofinite_imp_EI` and its mirror `ascent_cofinite_imp_EI`.
8. **BandBoundedAbove** — The bounded-above band recurrence `band_of_boundedAbove` (Theorem 3.12), packaged as `bandProvider_of_boundedAbove` and the bounded-above winner `notEI_boundedAbove_BobWins`.
9. **Q2Even** — The unbounded-above separator lemma `q2even` (Theorem 3.14), via the leftward-chain and residue-pigeonhole argument.
10. **GrowStair** — The growing-lag staircase (`gmoves`) and the unbounded-above winner `notEI_unboundedAbove_BobWins`.
11. **Answer** — The main theorem `main`, and the consistency capstone `locallyDecodable_main`.

4.2 The main theorem

The capstone, stated and proved in Lean 4, dispatches the winning direction on bounded-above:

```
theorem main (c : Perm) : BobWins c ↔ ¬ EventuallyIdentity c
```

The forward implication ($\rightarrow$) is the losing direction (`EventuallyIdentity_loses`). The reverse ($\leftarrow$) does a `by_cases` on whether c is bounded above ($\exists B, \forall n, (cn) \leq n + B$): the bounded-above branch calls `notEI_boundedAbove_BobWins` (the fixed-lag staircase fed by `band_of_boundedAbove`), and the unbounded-above branch calls `notEI_unboundedAbove_BobWins` (the growing-lag staircase fed by `q2even`). Both branches route through the axiom-clean reduction `localizes_BobWins`, so the whole proof is sorry-free.

4.3 Formalisation status

Component	Status	Lean name
Main <i>iff</i> (Thm. 1.1)	axiom-clean	main
Losing direction (Thm. 2.1)	axiom-clean	EventuallyIdentity_loses
Distinguishability (Thm. 3.2)	axiom-clean	Indistinguishable_implies_eventually_identity; not_eventually_identity_implies_distinguishable
Cut lemma (Lem. 3.13)	axiom-clean	descent_cofinite_imp_EI
First-Yes finiteness (Lem. 3.10)	axiom-clean	yesSet_finite
Reduction (Thm. 3.9)	axiom-clean	localizes_BobWins, ofMoves_localizes
Bounded-above band (Thm. 3.12)	axiom-clean	band_of_boundedAbove
Bounded-above winner	axiom-clean	notEI_boundedAbove_BobWins
Unbounded-above separators (Thm. 3.14)	axiom-clean	q2even
Unbounded-above winner	axiom-clean	notEI_unboundedAbove_BobWins
K -decodable winning classes (Thm. 3.17)	axiom-clean	LocallyDecodable_BobWins ($K=2$), LocallyK3Decodable_BobWins ($K=3$), LocallyKDecodable_BobWins (any K)
Decodable consistency capstone	axiom-clean	locallyDecodable_main

Table 2: Formalisation status. Every component—including the main *iff*—is machine-checked with no sorry; “axiom-clean” = the `#print axioms` audit lists only `propext`, `Classical.choice`, `Quot.sound`.

4.4 The journey, and what made it work

The proof was not found in this clean shape. Three milestones reshaped it. First, the distinguishability crux (Theorem 3.2, $\Delta \geq 2$) was long believed to need global surjectivity bookkeeping or a delicate even/odd case split; it fell to the elementary forward-closure argument (Lemma 3.4), which turns the active set on each residue into a finite prefix. Second, the winning direction was initially reduced to a single combinatorial “band recurrence” lemma believed true and computationally well-supported. That belief was *wrong*: the AP-split permutation (Remark 3.11) is an explicit counterexample, and recognising this was the turning point. Third, the band recurrence’s failure is *structural*, not accidental—it fails exactly when c is unbounded above—which yielded the clean dichotomy of Section 3.4. The bounded-above regime keeps the original fixed-lag staircase (now resting on the cut lemma rather than the false recurrence); the unbounded-above regime gets a growing-lag staircase fed by `q2even`, proved via a leftward-chain and residue-pigeonhole argument. Crucially, no adaptive belief-state strategy was ever needed—a fixed, c -tailored, signal-independent trajectory suffices in both regimes, which is precisely what the candidate-collapsing reduction (Theorem 3.9) consumes.

Acknowledgements

The solution to this puzzle — including the characterisation of Bob’s winning condition, the distinguishability theorem, the discovery that the band recurrence is false, and the bounded/unbounded staircase dichotomy — was developed in collaboration with [Claude](#) (Anthropic). The simulation and game-solver code, the Lean 4 formalisation, and this writeup were produced in collaboration with [Claude Code](#) (Anthropic).

References

- [1] The Mathlib Community (2020). *Lean 4 Mathematical Library* (MATHLIB4). <https://leanprover-community.github.io>.
- [2] de Moura, L. & Ullrich, S. (2021). The Lean 4 Theorem Prover and Programming Language. *CADE 28*, LNCS **12699**, 625–635.